Конашинская Ольга Александровна 090303-Пиб-о24

Лабораторная работа № 7

Вариант: № 10

Тема: Принципы SOLID при проектировании объектно-ориентированных приложений на Python. Рефакторинг программного кода и разделение ответственности между компонентами системы.

Постановка задачи:

Необходимо разработать приложение выбранной предметной области согласно индивидуальному варианту.

Этап 1. Реализация проблемной архитектуры

Сначала необходимо создать рабочую версию приложения, содержащую намеренные архитектурные проблемы.

Она должна содержать как минимум:

- класс с несколькими ответственностями;
- жёстко заданную реализацию зависимости;
- условную конструкцию, которую можно заменить полиморфизмом;
- чрезмерно большой интерфейс;
- хотя бы одну проблемную иерархию наследования.

После реализации студент должен самостоятельно указать, какие принципы SOLID нарушаются.

Этап 2. Рефакторинг

После создания первой версии необходимо переработать программу.

Вторая версия должна:

1. разделять ответственности классов;
2. использовать абстракции;
3. позволять добавлять новые реализации без изменения существующей бизнес-логики;
4. не содержать неправильного наследования;
5. использовать небольшие специализированные интерфейсы;

использовать Dependency Injection.

Приложение должно содержать:

Краткие теоретические сведения:

SOLID

- Определение: SOLID представляет собой систему из пяти фундаментальных принципов проектирования объектно-ориентированного программного обеспечения.
- Целевое назначение: Повышение декомпозиции, читаемости, тестируемости, адаптируемости и сопровождаемости программного кода.
- Характер требований: Принципы не являются синтаксическими правилами языка, а служат методологическими рекомендациями по управлению зависимостями и распределению зон ответственности.

Принципы SOLID

- S (Single Responsibility Principle): Каждый класс должен обладать единственной областью ответственности и только одним вектором потенциальных изменений.
- O (Open/Closed Principle): Программные компоненты проектируются открытыми для расширения функциональности, но закрытыми для модификации их исходного кода.
- L (Liskov Substitution Principle): Объекты подклассов должны сохранять корректность поведения программы при подстановке вместо объектов базового класса (соблюдение инвариантов и контрактов родительского класса).
- I (Interface Segregation Principle): Минимизация интерфейсов. Программные сущности не должны принудительно зависеть от методов интерфейса, которые ими не используются.
- D (Dependency Inversion Principle): Модули верхних уровней абстракции не должны иметь жестких зависимостей от конкретных реализаций низкого уровня. Взаимодействие должно строиться исключительно на основе абстракций.

Инструменты абстракции в Python

- Абстрактные базовые классы (abc.ABC): Применяются для явного задания интерфейсов и контрактов через механизм наследования.
- Протоколы (typing.Protocol): Используются для реализации структурной типизации («утиной типизации»), разделяя описание интерфейса и его конкретные реализации на уровне архитектуры без явного наследования.

Паттерн Dependency Injection (Внедрение зависимостей)

- Суть подхода: Исключение самостоятельного создания зависимых объектов внутри целевого класса; внешние компоненты поставляются в готовом виде извне.
- Классификация методов внедрения:
    - Constructor Injection (Внедрение через конструктор __init__) — является целевым стандартом для разработки.
    - Method Injection (Передача зависимости в качестве параметра метода).
    - Property/Setter Injection (Назначение зависимости через свойства объекта).

Исходный код программы:

main.py

```
from repositories import InMemoryTicketRepository
from notifier import ConsoleNotifier
from services import TicketService
from ui import ConsoleUI
from models import Train

def main() -> None:
    repository = InMemoryTicketRepository()
    notifier = ConsoleNotifier()
    service = TicketService(repository, notifier)

    trains = [
        Train(1, "001A", "Москва - Санкт-Петербург", 20),
        Train(2, "055B", "Москва - Казань", 16),
        Train(3, "120C", "Санкт-Петербург - Казань", 12),
    ]
```

```
    ui = ConsoleUI(service, trains)
    ui.run()

if __name__ == "__main__":
    main()
```

ui.py

```
from services import TicketService, TicketNotFoundError
from models import Train
from payments import CardPayment, CashPayment, BankTransfer


class ConsoleUI:
    def __init__(self, service: TicketService, trains: list[Train]) ->
None:
        self.service = service
        self.trains = trains

    def run(self) -> None:
        while True:
            print("===== ЖЕЛЕЗНОДОРОЖНЫЕ ПЕРЕВОЗКИ =====",
                  "1 - Поиск поезда",
                  "2 - Забронировать билет",
                  "3 - Купить (оплатить) билет",
                  "4 - Вернуть билет",
                  "5 - Показать все билеты",
                  "0 - Выход", sep="\n")
            choice = input("Выберите пункт меню: ")
            if choice == "1":
                self._search()
            elif choice == "2":
                self._book()
            elif choice == "3":
                self._buy()
            elif choice == "4":
                self._refund()
            elif choice == "5":
                self._show_all()
            elif choice == "0":
                print("Программа завершена!")
                break
            else:
                print("Некорректный ввод.")

    def _find_train(self, number: str) -> Train | None:
        for t in self.trains:
            if t.number == number:
```

```
                return t
        return None

    def _search(self) -> None:
        route = input("Введите направление: ")
        found = self.service.search_trains(self.trains, route)
        if found:
            for t in found:
                print(f"Поезд №{t.number}: {t.route}, мест:
{t.seats}")
        else:
            print("Поезда не найдены.")

    def _book(self) -> None:
        try:
            train = self._find_train(input("Номер поезда: "))
            if train is None:
                print("Поезд не найден.")
                return
            passenger = input("ФИО пассажира: ")
            seat = int(input("Место: "))
            price = float(input("Цена: "))
            ticket = self.service.book(train, passenger, seat, price)
            print(f"Забронирован билет #{ticket.id}")
        except ValueError:
            print("Ошибка: введено не число.")

    def _buy(self) -> None:
        methods = {"1": CardPayment(), "2": CashPayment(), "3":
BankTransfer()}
        try:
            ticket_id = int(input("ID билета: "))
            print("Способ оплаты: 1 - картой, 2 - наличными, 3 -
перевод")
            payment = methods.get(input("Выберите: "))
            if payment is None:
                print("Неизвестный способ оплаты.")
                return
            self.service.buy(ticket_id, payment)
            print("Билет оплачен.")
        except TicketNotFoundError as e:
            print("Ошибка:", e)
        except ValueError:
            print("Ошибка: введено не число.")

    def _refund(self) -> None:
        try:
            ticket_id = int(input("ID билета: "))
            self.service.refund(ticket_id)
            print("Билет возвращён.")
```

```
            except TicketNotFoundError as e:
                print("Ошибка:", e)
            except ValueError:
                print("Ошибка: введено не число.")

    def _show_all(self) -> None:
        tickets = self.service.repository.get_all()
        if not tickets:
            print("Билетов нет.")
        for t in tickets:
            print(f"#{t.id}: {t.passenger}, поезд {t.train_number},
место {t.seat} — {t.status.value}")
```

interface.py

```
from abc import ABC, abstractmethod
from typing import Protocol
from models import Ticket


# ISP: два небольших специализированных интерфейса вместо одного
большого
class TicketReader(Protocol):
    def get(self, ticket_id: int) -> Ticket | None:
        ...

    def get_all(self) -> list[Ticket]:
        ...

class TicketWriter(Protocol):
    def add(self, ticket: Ticket) -> None:
        ...

    def remove(self, ticket_id: int) -> None:
        ...

class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...

class PaymentMethod(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        ...
```

models.py

```
from dataclasses import dataclass
from enum import Enum


class TicketStatus(Enum):
    RESERVED = "забронирован"
    PAID = "оплачен"
    USED = "использован"
    RETURNED = "возвращён"

@dataclass
class Entity:
    id: int

@dataclass
class Train(Entity):
    number: str
    route: str
    seats: int

@dataclass
class Ticket(Entity):
    passenger: str
    train_number: str
    seat: int
    price: float
    status: TicketStatus = TicketStatus.RESERVED
```

notifier.py

```
class ConsoleNotifier:
    def send(self, recipient: str, message: str) -> None:
        print(f"[УВЕДОМЛЕНИЕ -> {recipient}] {message}")
```

payments.py

```
from interfaces import PaymentMethod

class CardPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Оплата картой: {amount} руб.")

class CashPayment(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Оплата наличными: {amount} руб.")
```

```
class BankTransfer(PaymentMethod):
    def pay(self, amount: float) -> None:
        print(f"Оплата банковским переводом: {amount} руб.")
```

problem_version.py

```
class Ticket:
    def __init__(self, passenger, train, seat, price):
        self.passenger = passenger
        self.train = train
        self.seat = seat
        self.price = price
        self.status = "забронирован"

    def refund(self):
        self.status = "возвращён"


class SeasonTicket(Ticket):             # нарушение LSP
    def refund(self):
        raise NotImplementedError("Абонемент вернуть нельзя")


class TicketService:                    # нарушение SRP и DIP
    def __init__(self):
        self.tickets = []               # хранилище жёстко внутри
класса

    def buy(self, passenger, train, seat, price, payment_type):
        ticket = Ticket(passenger, train, seat, price)
        self.tickets.append(ticket)
        # нарушение OCP - новый способ оплаты требует правки метода
        if payment_type == "card":
            print(f"Оплата картой: {price}")
        elif payment_type == "cash":
            print(f"Оплата наличными: {price}")
        elif payment_type == "transfer":
            print(f"Оплата переводом: {price}")
        else:
            print("Неизвестный способ оплаты")
        ticket.status = "оплачен"
        print(f"Билет для {passenger} оформлен")              # UI
внутри сервиса
        print(f"[EMAIL] Уведомление отправлено {passenger}")  #
уведомление внутри сервиса
        return ticket


class TicketManager:                    # нарушение ISP - слишком
```

```
большой интерфейс
    def add(self): ...
    def remove(self): ...
    def get(self): ...
    def print_report(self): ...
    def send_email(self): ...
    def send_sms(self): ...
    def export_pdf(self): ...


if __name__ == "__main__":
    service = TicketService()
    service.buy("Иванов И.И.", "001A", 5, 2500, "card")
```

repositories.py

```
from models import Ticket

class InMemoryTicketRepository:
    def __init__(self) -> None:
        self._items: list[Ticket] = []

    def add(self, ticket: Ticket) -> None:
        self._items.append(ticket)

    def remove(self, ticket_id: int) -> None:
        self._items = [t for t in self._items if t.id != ticket_id]

    def get(self, ticket_id: int) -> Ticket | None:
        for t in self._items:
            if t.id == ticket_id:
                return t
        return None

    def get_all(self) -> list[Ticket]:
        return list(self._items)
```

services.py

```
from models import Ticket, Train, TicketStatus
from interfaces import PaymentMethod, Notifier

class TicketNotFoundError(Exception):
    pass

class TicketService:
    def __init__(self, repository, notifier: Notifier) -> None:
        self.repository = repository          # DIP: зависимость
```

```
передаётся извне
        self.notifier = notifier
        self._next_id = 1

    def search_trains(self, trains: list[Train], route: str) ->
list[Train]:
        return [t for t in trains if route.lower() in t.route.lower()]

    def book(self, train: Train, passenger: str, seat: int, price:
float) -> Ticket:
        ticket = Ticket(self._next_id, passenger, train.number, seat,
price)
        self._next_id += 1
        self.repository.add(ticket)
        self.notifier.send(passenger, f"Билет на поезд {train.number}
забронирован")
        return ticket

    def buy(self, ticket_id: int, payment: PaymentMethod) -> None:
        ticket = self._get(ticket_id)
        payment.pay(ticket.price)             # OCP: способ оплаты
приходит извне
        ticket.status = TicketStatus.PAID
        self.notifier.send(ticket.passenger, "Билет оплачен")

    def refund(self, ticket_id: int) -> None:
        ticket = self._get(ticket_id)
        ticket.status = TicketStatus.RETURNED
        self.notifier.send(ticket.passenger, "Билет возвращён")

    def _get(self, ticket_id: int) -> Ticket:
        ticket = self.repository.get(ticket_id)
        if ticket is None:
            raise TicketNotFoundError(f"Билет #{ticket_id} не найден")
        return ticket
```

```
===== ЖЕЛЕЗНОДОРОЖНЫЕ ПЕРЕВОЗКИ =====
1 - Поиск поезда
2 - Забронировать билет
3 - Купить (оплатить) билет
4 - Вернуть билет
5 - Показать все билеты
0 - Выход
Выберите пункт меню: 1
Введите направление: Москва - Санкт-Петербург
Поезд №001А: Москва - Санкт-Петербург, мест: 20
```

```
Выберите пункт меню: 2
Номер поезда: 001А
ФИО пассажира: ФИО
Место: 1
Цена: 1500
[УВЕДОМЛЕНИЕ -> ФИО] Билет на поезд 001А забронирован
Забронирован билет #1
```

```
Выберите пункт меню: 3
ID билета: 1
Способ оплаты: 1 - картой, 2 - наличными, 3 - перевод
Выберите: 1
Оплата картой: 1500.0 руб.
[УВЕДОМЛЕНИЕ -> ФИО] Билет оплачен
Билет оплачен.
```

```
Выберите пункт меню: 4
ID билета: 1
[УВЕДОМЛЕНИЕ -> ФИО] Билет возвращён
Билет возвращён.
```

```
Выберите пункт меню: 5
#1: ФИО, поезд 001А, место 1 — возвращён
```